



Programa Certified AI-LLM Solution Architect

Curso Diseño de Fundamentos de Arquitectura LLM

APIs REST, FastAPI y LLMs locales con Ollama

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



NIVELACION 1b

SESION 1b

3 HORAS

Kubernetes, Docker y LLMs en la Nube

HTTP, FastAPI, Ollama y React — tu primer LLM Gateway completo



REST



GET



POST



JSON



SSE

Agenda — Sesión 1b

Tres bloques de trabajo · 3 horas · Construir un LLM Gateway completo

01

60 min

Bloque 1 — HTTP y APIs REST

- ▶ HTTP: GET, POST, headers, status codes
- ▶ Manejo de errores HTTP
- ▶ Streaming SSE: respuesta token a token
- ▶ React frontend con chat UI

02

60 min

Bloque 2 — Ollama API y Streaming

- ▶ requests: GET y POST con Python
- ▶ Ollama API: /api/chat, /api/generate
- ▶ Embeddings y similitud coseno
- ▶ Demo completo: React → FastAPI → Ollama

03

50 min

Bloque 3 — FastAPI + React + Demo

- ▶ JSON body, Bearer token, timeout
- ▶ OpenAI SDK compatible con Ollama
- ▶ FastAPI: rutas, Pydantic, CORS
- ▶ Streaming en el frontend



Break 10 min entre Bloque 1 y 2

¿Por que APIs REST para LLMs?

Tema 1 · Toda IA en produccion expone una API — debes saber hablarle

Sin APIs

- ✗ Cada LLM con interfaz diferente — no reutilizas codigo
- ✗ Logica mezclada con UI — dificil de escalar o reemplazar
- ✗ Sin documentacion automatica — cada dev reinventa la rueda
- ✗ Acoplado: cambiar el modelo rompe el cliente
- ✗ Sin streaming: usuario espera 30s sin feedback



Con REST API

- ✓ REST estandar: mismo patron para OpenAI, Anthropic, Ollama
- ✓ Backend independiente: cambia el LLM sin tocar el frontend
- ✓ FastAPI genera Swagger UI automatico en /docs
- ✓ API contract: frontend y backend evolucionan independientes
- ✓ SSE streaming: primer token en <1s, UX profesional



El 100% de los LLM providers (OpenAI, Anthropic, Cohere) exponen REST APIs con el mismo patron HTTP

HTTP: Anatomia de una Request/Response

Tema 1 · El protocolo que usan todos los LLMs — HTTP sobre TCP/IP



Request HTTP → Ollama

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

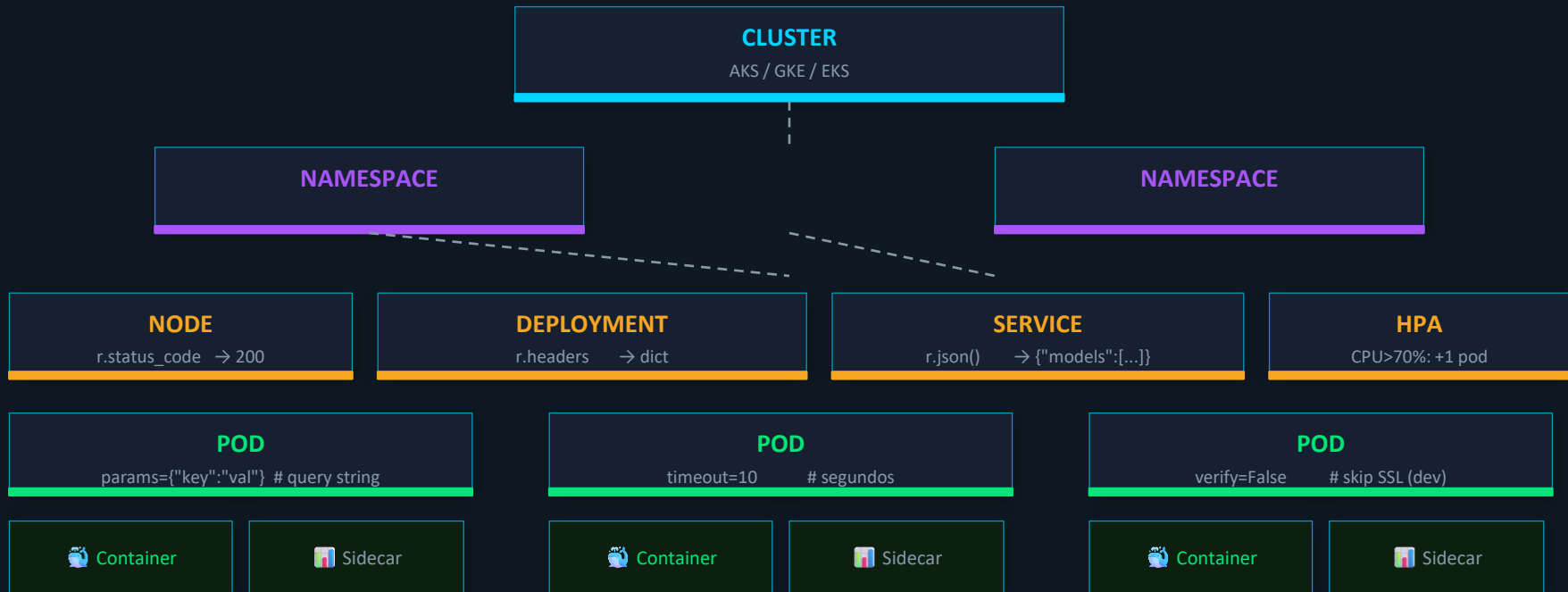
Status codes clave

```
200 OK      - éxito
401 Unauth  - sin API key
429 Too Many - rate limit
503 Unavail - Ollama no corre
```

```
400 Bad Req - payload invalido
404 Not Found - endpoint mal escrito
500 Error - fallo del servidor
```

GET Requests — Consultar APIs de LLM

Tema 1 · GET para inspeccionar, verificar y listar — base de toda integracion



POST + JSON — El Patron de los LLMs

PRACTICA

Tema 1 · POST envia datos — el 90% de llamadas a LLMs son POST + JSON



03_post_json.py

```
payload = {  
  
r = requests.post(  
  
json=payload → Content-Type:application/json
```

Embeddings y similitud coseno:

llama3.2:3b 2GB ★ Recomendado aula

mistral:7b 4GB Buenas instrucciones

phi3:mini 2.3GB Rápido y eficiente



Diferencias clave

"model":
"llama3.2",

"messages":
historial,

"stream": False}

"model":
"llama3.2",

json=payload,
timeout=60)

texto =
r.json()["message"]["content"]

data=payload
d → form-
urlencoded
(NO para
data=payload
LLMs)

headers={"Authorizat
ion": "Bearer sk-
..."}

timeout=(5, 60) → connect, read
timeout

d → form-
urlencoded
(NO para
data=payload
LLMs)

d → form-
urlencoded
(NO para
LLMs)

```
@app.post("/chat")  
async def chat(req: ChatRequest):  
    resp = await ollama_client.chat(  
        model=req.model,  
        messages=req.messages)  
    return ChatResponse(**resp)
```

Headers y Autenticacion en APIs LLM

Tema 1 · Bearer token, API keys y Content-Type — los 3 headers que siempre usas



Headers de request

```
"Content-Type": "application/json",
"User-Agent": "BSG-LLM-Client/1.0"}

Anthropic: x-api-key: sk-ant-...

from dotenv import load_dotenv

API_KEY = os.getenv("OPENAI_API_KEY")

# Jamas hardcodees: sk-abcd1234...

docker push \

# Jamas hardcodees: sk-abcd1234...
```



headers

```
{
  "headers": headers
}

headers=headers)
```



Auth por proveedor

```
"Authorization": f"Bearer {API_KEY}",

r = requests.post(url, json=payload,

Cohere: Authorization: Bearer...

Groq: Authorization: Bearer...

from dotenv import load_dotenv

API_KEY = os.getenv("OPENAI_API_KEY")

123.dkr.ecr.../llm-gateway:v1
```



Ollama auth (local)

```
Guardar keys seguro:
import os
load_dotenv() # lee .env
API_KEY = os.getenv("OPENAI_API_KEY")

...
```

API de Ollama — Endpoints Disponibles

Tema 2 · Ollama corre en localhost:11434 — API REST completa

Endpoint	Metodo	Descripcion	Ejemplo
/api/tags	GET	Listar modelos instalados	GET
curl localhost:11434/api/tags	/api/chat	POST	Chat multi-turno con historial
{"model":"llama3.2","messages":[...]}	/api/generate	POST	Completar texto (single turn)
{"model":"llama3.2","prompt":"..."}	/api/embeddings	POST	Vectores semanticos (RAG)
{"model":"nomic-embed-text","prompt":"..."}	/v1/chat/completions	POST	OpenAI-compatible (SDK drop-in)
{"model":"llama3.2","messages":[...]}	/api/pull	POST	Descargar modelo
{"name":"llama3.2"}	04_ollama_api.py	BASE_URL	localhost:11434

💡 Ollama es 100% gratuito y corre localmente — sin API key, sin quota, sin costo. Ideal para desarrollo y clase

OpenAI SDK — Compatible con Ollama (y todos los LLMs)

Tema 2 · Un SDK, multiples proveedores — cambia base_url y listo

05_openai_compatible.py

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-gateway
  namespace: llm-prod
spec:
  replicas: 2
  selector:
    matchLabels:
      app: llm-gateway
  template:
    spec:
      containers:
        - name: llm-gateway
          image: REGISTRY/llm-gateway:v1
          ports:
```

Cambiar de proveedor:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
```

Extraccion JSON estructurada:

```
resp = client.chat.completions.create(
    temperature=0.0)
```

```
response_for_data["type":"j
son-objects",res
p.choices[0].m
essage.conten
t)
```

Streaming SSE — Respuesta Token a Token

Tema 2 · Sin streaming la UX es mala — con SSE el usuario ve el texto aparecer



streaming
api
=True

Ollama

Activar streaming en Ollama

```
r = requests.post(URL+"/api/chat",  
json={..., "stream":True}, stream=True)
```



iter_lines()

requests

Rendimiento & Latencia

```
for linea in r.iter_lines(): data =  
json.loads(linea)
```



delta
content

SDK

Extraer el texto del delta

```
token = data["message"]["content"] #  
puede ser ""
```



on_token()

SDK

Seguridad & Cumplimiento

```
if not data.get("done"): print(token,  
end="", flush=True)
```



06_streaming.py

stream=True en la llamada

```
for c in  
client.chat.completions.create(stream=True,  
...):  
print(c.choices[0].delta.content)
```



SDK version:

Soporte & SLA

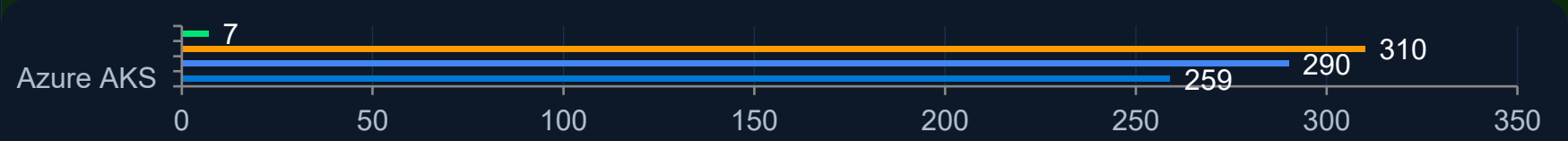
```
if c.choices[0].finish_reason == "stop":  
break
```



FastAPI — El Backend del LLM Gateway

Tema 3 · FastAPI genera tu API REST con Swagger UI, CORS y validacion automatica

Elemento	FastAPI	Python equiv.		
@app.get/post	Ruta HTTP	@router.get("/chat")		
BaseModel		\$73	\$73	
Schema Pydantic	class ChatRequest(BaseModel):			
Field()	Validacion auto	temperature: float = Field(0.7,ge=0,le=2)		
CORSMiddleware	Habilitar CORS	Validacion auto	Habilitar CORS	
async def	Handler asincrono	async def chat(req: ChatRequest):	temperature: float = Field(0.7,ge=0,le=2)	
HTTPException	Error con codigo HTTP	raise HTTPException(503,		



Pydantic — Validacion y Documentacion Gratis

Tema 3 · Define el schema una vez — FastAPI genera validacion + Swagger + tipos

Models del backend:

8000/
docs

Swagger UI

Auto

Tipos validados

Swagg
er

Errores claros

```
model: str = "llama3.2"
```

```
max_tokens: int = 1000
```

JSON
serializati
on
automatic

Error 422 si payload invalido

Beneficios automaticos:

class

Mensaje(BaseModel):

role: str

content: str

class ChatRequest(BaseModel):

messages:

list[Mensaje]

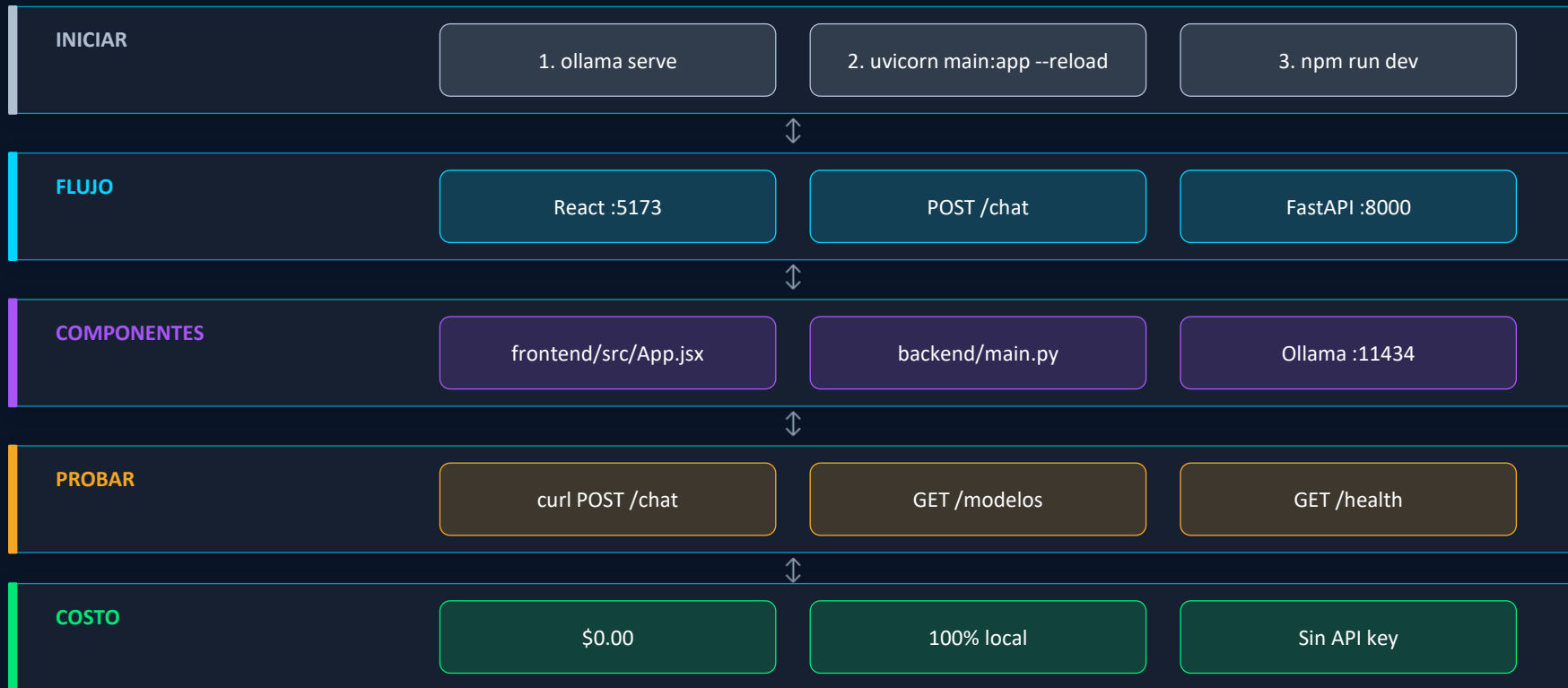
temperature: float = 0.7

Swagger gratis en /docs

Validacion automatica de tipos

Demo — React + FastAPI + Ollama

Stack completo corriendo en tu maquina — sin cloud, sin costo



Resumen — Lo que Aprendiste Hoy

De HTTP al LLM Gateway completo

Concepto

HTTP GET/POST

Ollama API

OpenAI SDK

Streaming SSE

FastAPI + Pydantic



Lo que sabes hacer

Inspeccionar APIs y manejar errores

Chat, generate, embeddings locales

Mismo código, cualquier proveedor LLM

Token a token — UX profesional

Backend REST con validación y Swagger



Siguiente sesión: Sesión 2 — RAG, Embeddings y LLM en Producción con Vector Databases

Recursos y Siguietes Pasos

TAREA

Antes de la proxima sesion — instalar y ejecutar el stack completo



Instalar y verificar:

```
pip install fastapi uvicorn httpx requests openai
```

```
node --version → v18+ requerido
```

```
uvicorn main:app → http://localhost:8000/docs
```

```
npm run dev → http://localhost:5173
```

```
session_1b/ejemplos/ → 6 scripts
```

```
session_1b/backend/ → FastAPI completo
```

```
session_1b/frontend/ → React chat UI
```

Preguntas frecuentes:

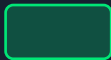
llama3.2:3b 2GB ★ Recomendado aula

mistral:7b 4GB Buenas instrucciones

phi3:mini 2.3GB Rápido y eficiente



Explorar el repositorio:



```
@app.post("/chat")
async def chat(req: ChatRequest):
    resp = await ollama_client.chat(
        model=req.model,
        messages=req.messages)
    return ChatResponse(**resp)
```



Resumen de la Sesión 1b



HTTP: GET/POST, status codes, headers, JSON body



Ollama API: /api/chat, /api/generate,
/api/embeddings



OpenAI SDK: base_url para Ollama, Groq, OpenAI



Streaming SSE: respuesta token a token desde el LLM



FastAPI + Pydantic: backend con validación y Swagger



React frontend: chat UI conectado al LLM Gateway

Recursos del Curso

Repositorio: github.com/bsg-institute/nivelacion-llm

Frontend: [session_1b/frontend/](#) (React + Vite)

Backend: [session_1b/backend/](#) (FastAPI)

Docs: [session_1b/docs/SETUP.md](#)